

Midlands Theatre Company Datamart Design and Analysis

Neha Vadapally



Table Of Contents

1. Star Schema Design	2
1.1 Star schema	2
1.2 Explanation of Star Schema.....	2
1.2.1 What are the fact and dimensions and why they are chosen	3
1.2.2 Why their attributes are chosen	3
1.2.3 What is the granularity	3
2. Logical Relations.....	3
2.1 List of logical Relations	4
2.2 Mapping from Star Schema to Logical Relations.....	4
3. Table Creation with Integrity Rules	5
3.1 Oracle SQL Code	5
3.2 Evidence	6
4. Data source Identification, data extraction, transformation and loading.....	7
4.1 Data source mapping	7
4.2 ETL Code	8
4.3 Evidence of ETL Code Implementation	10
5. Justification and Comparison	14
5.1 SQL code for Analysis.....	14
5.2 Evidence of SQL Code Implementation	16
5.3 Comparison of DataMart and OLTP	19
References.....	20

1. Star Schema Design

Midlands Theatre operates in major Midland's cities such as Birmingham and Leicester, offering alternative productions with matinee and evening shows. MT sells tickets using a computerised booking system, which maintains details about its clients, processes the sales, and manages information on all performance schedules. Although this OLTP system meets the day-to-day needs of MT for the processing and updating of sales information, it is less useful when detailed analysis is required. The aim of this report is to develop a data mart to meet these analytical requirements of MT. A star schema has been designed to determine the total sales by production, monthly sales by each theatre, and top-spending clients at each theatre, as shown in Figure 1.

1.1 Star schema

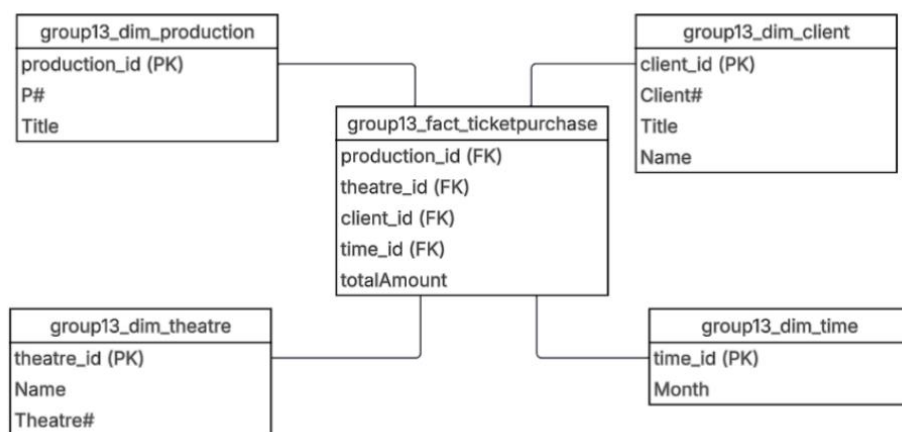


Figure 1: Start Schema Design (diagram created by authors using Lucid chart)

1.2 Explanation of Star Schema

The proposed star schema is a simple, single-fact design to achieve shorter query times and ease of understanding for analysis. Each element of the schema is designed to respond to three important MT business questions:

- Total sales by production: How much money does each production bring in?
- Monthly Sales by Theatre: What is the monthly sales number for each of the theatre?
- Top-spending clients by theatre: Which clients spend the most money at each theatre?

This setup lets the theatre analyse ticket sales and produce clear revenue and show-performance reports.

1.2.1 What are the fact and dimensions and why they are chosen

The proposed star schema consists of one central fact table and four supporting dimension tables. The fact table records each ticket purchase, which is the most detailed event MT needs to analyze. It includes the monetary measure of **TotalAmount**, which is essential for evaluating revenue and client spending.

The **dimension tables** provide descriptive context for analysis:

- **Production Dimension** was chosen to allow analysis by production title, so we can use it for questions about revenue per production.
- **Theatre Dimension** was included to allow location-based analysis, such as monthly sales by theatre.
- **Client Dimension** was selected to identify and rank clients based on their spending patterns.
- **Time Dimension** was added to allow trend analysis over time, such as monthly or yearly sales.

1.2.2 Why their attributes are chosen

Each attribute in the fact and dimension tables was chosen to enable filtering, grouping, and aggregation without redundancy.

- **Fact Table Attributes:**
 - production_id, theatre_id, client_id, and time_id are foreign keys that link the fact table to its dimensions, enabling multi-perspective analysis.
 - TotalAmount is the core **numeric measure** representing the value of each ticket purchase.
- **Dimension Table Attributes:**
 - **Production Dimension:** Includes production_id, P# and Title to uniquely identify and describe each production.
 - **Theatre Dimension:** Contains theatre_id, Theatre#, and Name to identify theatres and support location-based reporting.
 - **Client Dimension:** Stores client_id, Client#, Title and Name to identify clients and analyse spending behaviour.
 - **Time Dimension:** Includes time_id, year and month to enable time-based aggregation and trend analysis.

1.2.3 What is the granularity

The granularity of the fact table is defined at the level of an individual ticket purchase. Each record represents one ticket bought by one client for one production at one theatre on a specific date.

This level of detail allows flexible aggregation across multiple dimensions, such as monthly sales by theatre or total revenue by production. Kimball (2008) says that the need to define granularity clearly for accurate reporting and performance optimization.

2. Logical Relations

2.1 List of logical Relations

Based on the star schema design for the Ticket Sales Data Mart, the logical relations are derived as follows:

- 1) group13_dim_production (production_id, P#, Title)
- 2) group13_dim_theatre (theatre_id, Theatre#, Name)
- 3) group13_dim_client (client_id, Client#, Title, Name)
- 4) group13_dim_time (time_id, Year, Month)
- 5) group13_fact_ticketPurchase (production_id, theatre_id, client_id, time_id, TotalAmount)

2.2 Mapping from Star Schema to Logical Relations

- a. Each table in the star schema represents its corresponding entity and is named the same as the entity it models. For example,
 - group13_dim_production represents the group13_dim_production, group13_dim_theatre represents the group13_dim_theatre entity, group13_dim_client represents the group13_dim_client entity, and group13_dim_time represents the group13_dim_time entity.
 - The fact table group13_fact_ticketPurchase represents the ticket sales process and connects all these entities through foreign keys.
- b. A star schema uses surrogate keys such as production_id, theatre_id, client_id, and time_id instead of natural keys to provide stable and consistent identifiers in each dimension table. These surrogate keys together provide a composite primary key that relates each transaction to its related dimension of production, theatre, client, and time. They then serve as foreign keys in the fact table. Since the original one-to-many linkages have now been reduced to these foreign key links, the fact table can bring all the dimensions together and enable data analysis from any perspective. This approach enhances performance due to greater flexibility by reducing redundancy and maintaining organised data (Adamson, 2010; Connolly and Begg, 2015; Kimball et al., 2008). See Table 1 for the primary and foreign keys within the dimension and fact tables.

Table Name	Primary Key (PK)	Foreign Key (FK)
group13_dim_production	production_id	None
group13_dim_theatre	theatre_id	None
group13_dim_client	client_id	None
group13_dim_time	time_id	None
group13_fact_ticketpurchase	(production_id, theatre_id, client_id, time_id)	production_id → group13_dim_production(production_id) theatre_id → group13_dim_theatre(theatre_id) client_id → group13_dim_client(client_id) time_id → group13_dim_time(time_id)

Table 1: Entity and Referential Integrity for Dimension and Fact Tables

3. Table Creation with Integrity Rules

The entity rule in the dimensional modelling can be reflected in using primary keys in each dimension table that guarantees uniqueness and integrity for each record of that table as specified by Connolly and Begg (2015). In this schema, the referential rule applies to the fact table with the foreign keys that refer to the primary keys located in the dimension tables, ensuring that every fact record is tied to valid and meaningful dimension data in the words of Kimball et al. (2008). Dimension tables do not use foreign keys; each is supposed to be independent of descriptive information and leaving them that way simplifies the star schema and ensures better performance of queries. These rules are shown in Table 1, and their implementation code is provided below.

3.1 Oracle SQL Code

- **Dimension 1:** Group13_dim_production

```
CREATE TABLE group13_dim_production (  
  production_id NUMBER (5) PRIMARY KEY,  
  P# NUMBER (5) NOT NULL,  
  Title VARCHAR2(20) NOT NULL);
```

- **Dimension 2:** Group13_dim_theatre

```
CREATE TABLE group13_dim_theatre (  
  theatre_id NUMBER (5) PRIMARY KEY,  
  Theatre# NUMBER (5) NOT NULL,  
  Name VARCHAR2(20) NOT NULL);
```

- **Dimension 3:** Group13_dim_client

```
CREATE TABLE group13_dim_Client (  
  client_id NUMBER (5) PRIMARY KEY,  
  Client# NUMBER (5) NOT NULL,  
  Title VARCHAR2(10),  
  Name VARCHAR2(30) NOT NULL);
```

- **Dimension 4:** Group13_dim_time

```
CREATE TABLE group13_dim_time (  
  time_id NUMBER PRIMARY KEY,  
  Year NUMBER (4) NOT NULL,  
  Month NUMBER (2) NOT NULL);
```

- **Fact:** Group13_fact_ticketPurchase

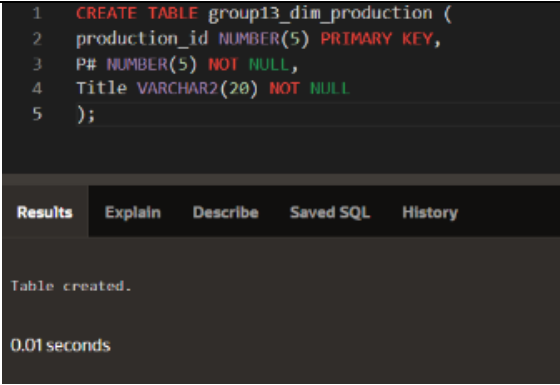
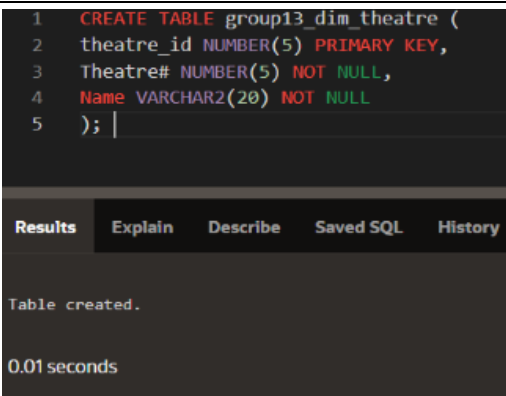
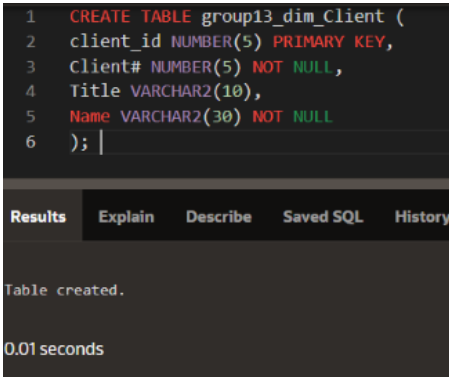
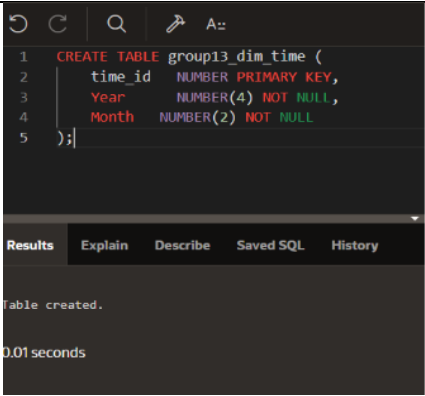
```
CREATE TABLE group13_fact_TicketPurchase (  
  production_id NUMBER(5)  
  CONSTRAINT fk_group13_dim_production REFERENCES group13_dim_production(production_id),  
  theatre_id NUMBER(5)  
  CONSTRAINT fk_group13_dim_theatre REFERENCES group13_dim_theatre(theatre_id),  
  client_id NUMBER(5)  
  CONSTRAINT fk_group13_dim_client REFERENCES group13_dim_client(client_id),  
  time_id NUMBER(5)
```

```

CONSTRAINT fk_group13_dim_time REFERENCES group13_dim_time(time_id),
TotalAmount NUMBER(10,2),
CONSTRAINT pk_group13_fact_TicketPurchase
PRIMARY KEY (production_id, theatre_id, client_id, time_id) );

```

3.2 Evidence

 group13_dim_production	 group13_dim_theatre
 group13_dim_Client	 group13_dim_time

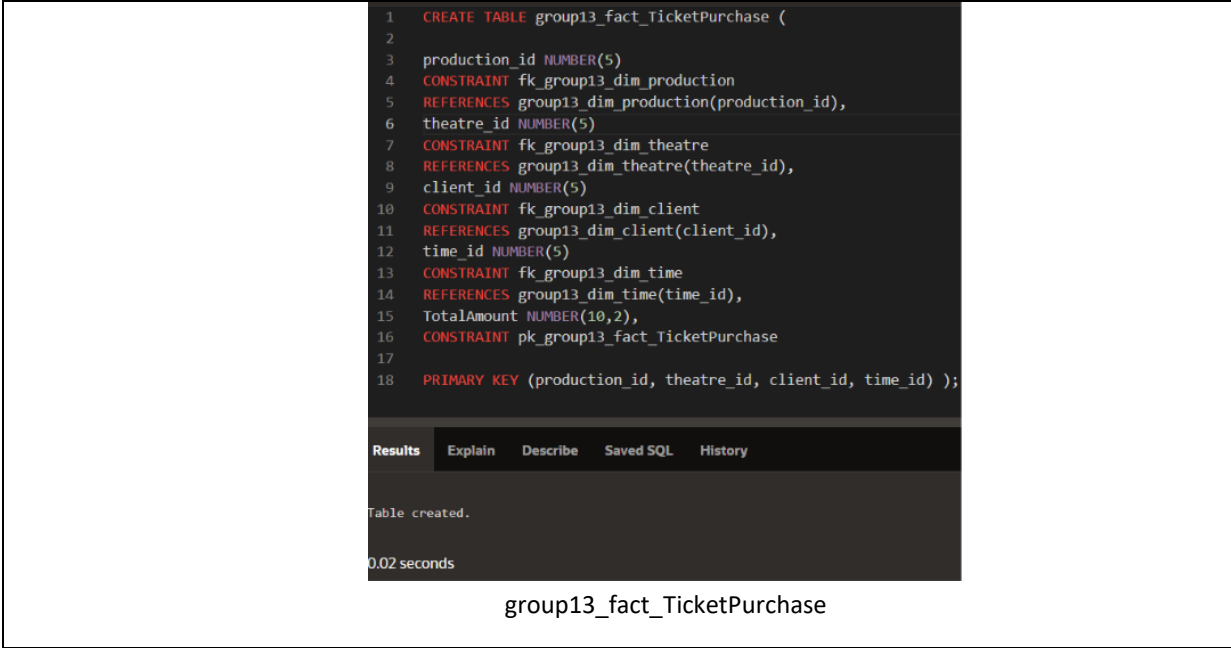


Table 2: Oracle SQL Dimension and Fact Table Creation

4. Data source Identification, data extraction, transformation and loading

4.1 Data source mapping

Dimension Production

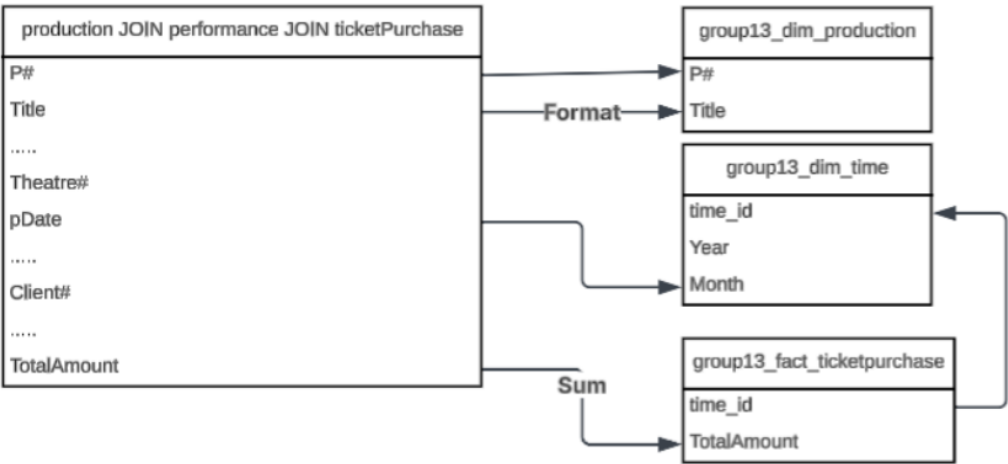


Figure 2: Data source mapping for Production dimension showing joins, formatting, and aggregation into dimension and fact tables. (diagram created by authors using Lucid chart)

Theatre Dimension

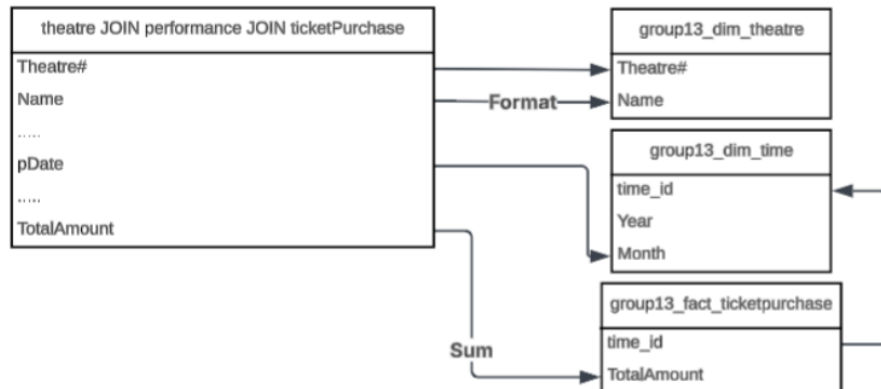


Figure 3: Data source mapping for theatre dimension illustrating transformation and linkage to time and fact tables. (diagram created by authors, using Lucid chart)

Client Dimension

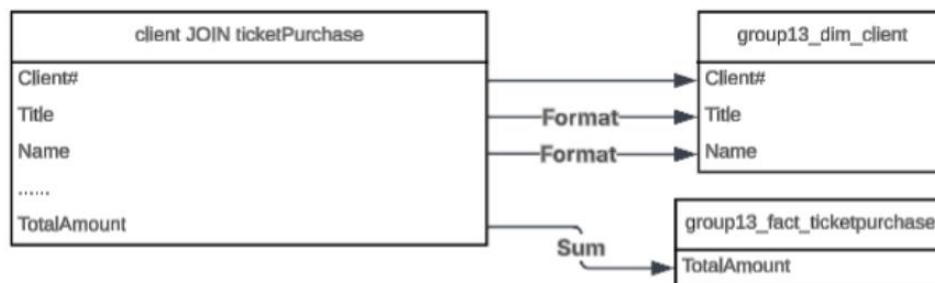


Figure 4: Data source mapping for client dimension depicting attribute formatting and fact table aggregation. (diagram created by authors, using Lucid chart)

4.2 ETL Code

This data mart includes data from ticket sales only. All the data not directly related to at least one ticket sale has been excluded from this data mart. In this way, only the relevant information will make the analysis faster and smaller scope. The ETL code below shows this process, plus a short summary table explaining each step of the ETL for the dimension and fact tables based on this requirement.

1) Production Dimension

- CREATE SEQUENCE seq_group13_dim_production START WITH 1 INCREMENT BY 1;
- INSERT INTO group13_dim_production
SELECT seq_group13_dim_production.NEXTVAL AS production_id,
p."P#" AS "P#",
INITCAP(TRIM(p.title)) AS Title
FROM (SELECT DISTINCT p."P#", p.title
FROM ops\$yyang00.production p

```

JOIN ops$yyang00.performance pf ON p."P#" = pf."P#"
JOIN ops$yyang00.ticketpurchase tp ON pf."PER#" = tp."PER#" ) p;

```

2) Theatre Dimension

- CREATE SEQUENCE seq_group13_dim_theatre START WITH 1 INCREMENT BY 1;
- INSERT INTO group13_dim_theatre


```

SELECT seq_group13_dim_theatre.NEXTVAL AS theatre_id,
       t."THEATRE#" AS "THEATRE#",
       INITCAP(TRIM(t.name)) AS Name
FROM ( SELECT DISTINCT t."THEATRE#", t.name
      FROM ops$yyang00.theatre t
      JOIN ops$yyang00.performance pf ON t."THEATRE#" = pf."THEATRE#"
      JOIN ops$yyang00.ticketpurchase tp ON pf."PER#" = tp."PER#" ) t;

```

3) Client Dimension

- CREATE SEQUENCE seq_group13_dim_client START WITH 1 INCREMENT BY 1;
- INSERT INTO group13_dim_client


```

SELECT seq_group13_dim_client.NEXTVAL AS client_id,
       c."CLIENT#" AS "CLIENT#",
       UPPER(TRIM(c.title)) AS Title,
       INITCAP(TRIM(c.name)) AS Name
FROM ( SELECT DISTINCT c."CLIENT#", c.title, c.name
      FROM ops$yyang00.client c
      JOIN ops$yyang00.ticketpurchase tp ON c."CLIENT#" = tp."CLIENT#" ) c;

```

4) Time Dimension

- CREATE SEQUENCE seq_group13_dim_time START WITH 1 INCREMENT BY 1;
- INSERT INTO group13_dim_time


```

SELECT seq_group13_dim_time.NEXTVAL, Year, Month
FROM ( SELECT DISTINCT EXTRACT(YEAR FROM pf.pdate) AS Year,
                      EXTRACT(MONTH FROM pf.pdate) AS Month
      FROM ops$yyang00.performance pf
      JOIN ops$yyang00.ticketpurchase tp ON pf."PER#" = tp."PER#"
      ) pf;

```

5) Ticket Purchase Fact

- INSERT INTO group13_fact_ticketPurchase
SELECT p.production_id, th.theatre_id, c.client_id, tm.time_id, SUM(tp.totalamount)
AS TotalAmount FROM ops\$yyang00.ticketpurchase tp
JOIN ops\$yyang00.performance pf ON tp."PER#" = pf."PER#"
JOIN group13_dim_production p ON pf."P#" = p."P#"
JOIN group13_dim_theatre th ON pf."THEATRE#" = th."THEATRE#"
JOIN group13_dim_client c ON tp."CLIENT#" = c."CLIENT#"
JOIN group13_dim_time tm
ON EXTRACT(YEAR FROM pf.pdate) = tm.Year
AND EXTRACT(MONTH FROM pf.pdate) = tm.Month
GROUP BY p.production_id, th.theatre_id, c.client_id, tm.time_id;

Table	Extract	Transform	Load
Production Dimension	Extracts productions whose performances have been purchased.	Cleans production names using TRIM to remove extra spaces and INITCAP to standardise the format.	Loads the cleaned data into group13_dim_production and assigns a surrogate key.
Theatre Dimension	Extracts theatres where performances have been purchased.	Standardises theatre names using TRIM and INITCAP.	Loads the results into group13_dim_theatre and creates a surrogate key for each theatre.
Client Dimension	Extracts clients who have purchased tickets for a performance.	Cleans the data using TRIM, formats titles with UPPER, and standardises names using INITCAP.	Loads the cleaned client data into group13_dim_client and assigns a surrogate key.
Time Dimension	Extracts performance dates linked to ticket purchases.	Uses EXTRACT(YEAR) and EXTRACT(MONTH) to pull out the year and month.	Loads the values into group13_dim_time and assigns a surrogate key.
Fact	Extracts ticket purchase and performance data and links them with all dimension keys.	Calculates total sales using SUM (totalamount) for each production, theatre, client, and time combination.	Loads the final aggregated results into group13_fact_ticketPurchase.

Table 3: ETL Summary Table

4.3 Evidence of ETL Code Implementation

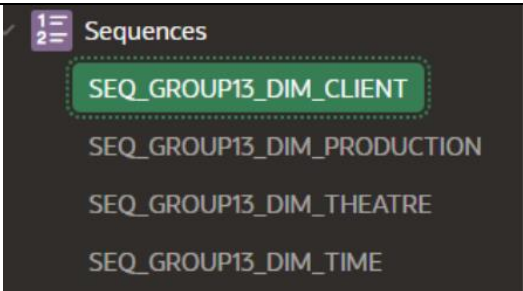
<pre>SQL CREATE SEQUENCE seq_group13_dim_production START WITH 1 INCREMENT BY 1; CREATE SEQUENCE seq_group13_dim_time START WITH 1 INCREMENT BY 1; CREATE SEQUENCE seq_group13_dim_client START WITH 1 INCREMENT BY 1; CREATE SEQUENCE seq_group13_dim_theatre START WITH 1 INCREMENT BY 1;</pre>	
---	--

Table 4: Snippets for Sequence Creation for Dimension Tables

<pre>1 INSERT INTO group13_dim_production 2 SELECT seq_group13_dim_production.NEXTVAL AS production_id, 3 p."P#" AS "P#", 4 INITCAP(TRIM(p.title)) AS Title 5 FROM (6 SELECT DISTINCT p."P#", p.title 7 FROM ops\$yyang00.production p 8 JOIN ops\$yyang00.performance pf ON p."P#" = pf."P#" 9 JOIN ops\$yyang00.ticketpurchase tp ON pf."PER#" = tp."PER#" 10) p;</pre>
Results Explain Describe Saved SQL History 78 row(s) inserted. 0.02 seconds
<pre>1 select count(*) as NO_OF_ROWS_INSERTED_DIM_PRODUCTION from group13_dim_production;</pre>
Results Explain Describe Saved SQL History NO_OF_ROWS_INSERTED_DIM_PRODUCTION 78 1 rows returned in 0.00 seconds Download

Table 5: Snippets for ETL Process of production dimension

```

1  INSERT INTO group13_dim_theatre
2  SELECT seq_group13_dim_theatre.NEXTVAL AS theatre_id,
3  t."THEATRE#" AS "THEATRE#",
4  INITCAP(TRIM(t.name)) AS Name
5  FROM (
6    SELECT DISTINCT t."THEATRE#", t.name
7    FROM ops$yyang00.theatre t
8    JOIN ops$yyang00.performance pf ON t."THEATRE#" = pf."THEATRE#"
9    JOIN ops$yyang00.ticketpurchase tp ON pf."PER#" = tp."PER#"
10   ) t;

```

Results Explain Describe Saved SQL History

6 row(s) inserted.

0.02 seconds

```

1  select count(*) AS NO_OF_ROWS_INSERTED_DIM_THEATRE from group13_dim_theatre;

```

Results Explain Describe Saved SQL History

Bottom Splitter

NO_OF_ROWS_INSERTED_DIM_THEATRE
6

1 rows returned in 0.00 seconds [Download](#)

Table 6: Snippets for ETL Process of theatre dimension

```
1  INSERT INTO group13_dim_client
2  SELECT seq_group13_dim_client.NEXTVAL AS client_id,
3  c."CLIENT#" AS "CLIENT#",
4  UPPER(TRIM(c.title)) AS Title,
5  INITCAP(TRIM(c.name)) AS Name
6  FROM (
7      SELECT DISTINCT c."CLIENT#", c.title, c.name
8      FROM ops$yyang00.client c
9      JOIN ops$yyang00.ticketpurchase tp ON c."CLIENT#" = tp."CLIENT#"
10 ) c;
```

Results

Explain

Describe

Saved SQL

History

3090 row(s) inserted.

0.11 seconds

```
1  select count(*) as NO_OF_ROWS_INSERTED_DIM_CLIENT from group13_dim_client;
```

Results

Explain

Describe

Saved SQL

History

NO_OF_ROWS_INSERTED_DIM_CLIENT
3090

1 rows returned in 0.01 seconds [Download](#)

Table 7: Snippets for ETL Process of client dimension

1	INSERT INTO group13_dim_time
2	SELECT seq_group13_dim_time.NEXTVAL, Year, Month
3	FROM (
4	SELECT DISTINCT EXTRACT(YEAR FROM pf.pdate) AS Year,
5	EXTRACT(MONTH FROM pf.pdate) AS Month
6	FROM ops\$yyang00.performance pf
7	JOIN ops\$yyang00.ticketpurchase tp ON pf."PER#" = tp."PER#"
8	) pf;
Results Explain Describe Saved SQL History	
17 row(s) inserted.	
0.01 seconds	
1	SELECT COUNT(*) AS NO_OF_ROWS_INSERTED_DIM_TIME FROM group13_dim_time;
Results Explain Describe Saved SQL History	
NO_OF_ROWS_INSERTED_DIM_TIME	
17	
1 rows returned in 0.01 seconds Download	

Table 8: Snippets for ETL Process of time dimension

1	INSERT INTO group13_fact_ticketPurchase
2	SELECT p.production_id, th.theatre_id, c.client_id, tm.time_id,
3	SUM(tp.totalamount) AS TotalAmount
4	FROM ops\$yyang00.ticketpurchase tp
5	JOIN ops\$yyang00.performance pf ON tp."PER#" = pf."PER#"
6	JOIN group13_dim_production p ON pf."P#" = p."P#"
7	JOIN group13_dim_theatre th ON pf."THEATRE#" = th."THEATRE#"
8	JOIN group13_dim_client c ON tp."CLIENT#" = c."CLIENT#"
9	JOIN group13_dim_time tm
10	ON EXTRACT(YEAR FROM pf.pdate) = tm.Year
11	AND EXTRACT(MONTH FROM pf.pdate) = tm.Month
12	GROUP BY p.production_id, th.theatre_id, c.client_id, tm.time_id;
13	
Results Explain Describe Saved SQL History	
4813 row(s) inserted.	
0.19 seconds	
1	select count(*) as NO_OF_ROWS_INSERTED_TICKETPURCHASE from group13_fact_ticketpurchase;
Results Explain Describe Bottom Splitter tory	
NO_OF_ROWS_INSERTED_TICKETPURCHASE	
4813	
rows returned in 0.00 seconds Download	

Table 9: Snippets for ETL Process of fact ticket purchase

5. Justification and Comparison

5.1 SQL code for Analysis

i. Total sales by production

a. Data Mart

```
SELECT p.title AS Production,  
       SUM(f.TotalAmount) AS Total_Sales  
FROM group13_fact_ticketpurchase f  
JOIN group13_dim_production p ON f.production_id = p.production_id  
GROUP BY p.title  
ORDER BY Total_Sales DESC;
```

b. OLTP

```
SELECT p.title AS OLTP_Production,  
       SUM (tp.totalamount) AS OLTP_Total_Sales  
FROM ops$yyang00.production p  
JOIN ops$yyang00.performance pf ON p."P#" = pf."P#"  
JOIN ops$yyang00.ticketpurchase tp ON pf."PER#" = tp."PER#"  
GROUP BY p.title  
ORDER BY OLTP_Total_Sales DESC;
```

ii. Monthly Sales by Theatre

a. Data Mart

```
SELECT t.name AS Theatre, tm.Year,  
       TO_CHAR(TO_DATE(tm.Month, 'MM'), 'FMMonth') AS Month,  
       SUM(f.TotalAmount) AS Monthly_Sales  
FROM group13_fact_ticketPurchase f  
JOIN group13_dim_theatre t ON f.theatre_id = t.theatre_id  
JOIN group13_dim_time tm ON f.time_id = tm.time_id  
GROUP BY t.name, tm.Year, tm.Month  
ORDER BY tm.Year, tm.Month;
```

b. OLTP

```
SELECT t.name AS Theatre,  
       EXTRACT(YEAR FROM pf.pdate) AS Year,  
       TO_CHAR(pf.pdate, 'Month') AS Month,  
       SUM(tp.totalamount) AS Monthly_Sales  
FROM ops$yyang00.theatre t  
JOIN ops$yyang00.performance pf ON t."THEATRE#" = pf."THEATRE#"  
JOIN ops$yyang00.ticketpurchase tp ON pf."PER#" = tp."PER#"  
GROUP BY t.name, EXTRACT(YEAR FROM pf.pdate), TO_CHAR(pf.pdate, 'Month')  
ORDER BY Year, Month;
```

iii. Top-spending clients by theatre

a. Data Mart

```
SELECT t.name AS Theatre, (c.title || ' ' || c.name) AS Client, SUM(f.TotalAmount)
AS Total_Spent
FROM group13_fact_ticketpurchase f
JOIN group13_dim_theatre t ON f.theatre_id = t.theatre_id
JOIN group13_dim_client c ON f.client_id = c.client_id
GROUP BY t.theatre_id, t.name, c.title, c.name
HAVING SUM(f.TotalAmount) = (
    SELECT MAX(SUM(f2.TotalAmount))
    FROM group13_fact_ticketpurchase f2
    WHERE f2.theatre_id = t.theatre_id
GROUP BY f2.client_id )
ORDER BY t.name;
```

b. OLTP

```
SELECT t.name AS Theatre, c.name AS Client, SUM(tp.totalamount) AS Total_Spent
FROM ops$yyang00.theatre t
JOIN ops$yyang00.performance pf ON t."THEATRE#" = pf."THEATRE#"
JOIN ops$yyang00.ticketpurchase tp ON pf."PER#" = tp."PER#"
JOIN ops$yyang00.client c ON tp."CLIENT#" = c."CLIENT#"
GROUP BY t."THEATRE#", t.name, c.name
HAVING SUM(tp.totalamount) = (
    SELECT MAX(client_total)
    FROM ( SELECT tp2."CLIENT#", SUM(tp2.totalamount) AS client_total
    FROM ops$yyang00.ticketpurchase tp2
    JOIN ops$yyang00.performance pf2 ON tp2."PER#" = pf2."PER#"
    WHERE pf2."THEATRE#" = t."THEATRE#"
    GROUP BY tp2."CLIENT#"
)) ORDER BY t.name;
```

c. Index creation

```
CREATE INDEX idx_fact_analysis_theatre_client
ON group13_fact_ticketpurchase (theatre_id, client_id, TotalAmount);
```

5.2 Evidence of SQL Code Implementation

i. Total sales by production

a. Data Mart

```
1 SELECT
2     p.title AS Production,
3     SUM(f.TotalAmount) AS Total_Sales
4 FROM group13_fact_ticketpurchase f
5 JOIN group13_dim_production p
6     ON f.production_id = p.production_id
7 GROUP BY p.title
8 ORDER BY Total_Sales DESC;
```

Results	Explain	Describe	Saved SQL	History
Star war				520.5
Raining Day				520.5
Blue Sky				520.5
Friends				520
New World				520
Brothers				519.5
Morning Sunshine				437
Your Turn				430
Strike				351
Back To Home				346.5
Impossible				261.5
Lasting Friendship				260.5
Family				175
Father And Son				88
Old Man				88

77 rows returned in 0.01 seconds [Download](#)

Figure 5 Data Mart Analysis on Total Sales of each production

b. OLTP

```
1 SELECT
2     p.title AS OLTP_Production,
3     SUM(tp.totalamount) AS OLTP_Total_Sales
4 FROM ops$yyang00.production p
5 JOIN ops$yyang00.performance pf ON p."P#" = pf."P#"
6 JOIN ops$yyang00.ticketpurchase tp ON pf."PER#" = tp."PER#"
7 GROUP BY p.title
8 ORDER BY OLTP_Total_Sales DESC;
```

Results	Explain	Describe	Saved SQL	History
Blue sky				520.5
Star War				520.5
New World				520
Friends				520
Brothers				519.5
Morning Sunshine				437
Your Turn				430
Strike				351
Back to Home				346.5
Impossible				261.5
Lasting friendship				260.5
Family				175
Father and Son				88
Old Man				88

77 rows returned in 0.01 seconds [Download](#)

Figure 6 OLTP Analysis on Total Sales of each production

ii. Monthly Sales by Theatre

a. Data Mart

```

1 SELECT t.name AS Theatre, tm.Year,
2      TO_CHAR(TO_DATE(tm.Month, 'MM'), 'FMMonth') AS Month, SUM(f.TotalAmount) AS Monthly_Sales
3 FROM group13_fact_ticketPurchase f
4 JOIN group13_dim_theatre t ON f.theatre_id = t.theatre_id
5 JOIN group13_dim_time tm ON f.time_id = tm.time_id
6 GROUP BY t.name, tm.Year, tm.Month
7 ORDER BY tm.Year, tm.Month;

```

Results	Explain	Describe	Saved SQL	History
Cropston	2019	August	386.5	
Victory	2019	August	389	
Vue	2019	August	387.5	
Beeston	2019	September	375.5	
Birstall	2019	September	374.5	
Clinton	2019	September	372.5	
Cropston	2019	September	376	
Victory	2019	September	374	
Vue	2019	September	373.5	
Beeston	2019	October	298.5	
Birstall	2019	October	299	
Clinton	2019	October	298.5	
Cropston	2019	October	300	
Victory	2019	October	299.5	
Vue	2019	October	296	

102 rows returned in 0.02 seconds [Download](#)

Figure 7 Data Mart Analysis of Monthly sales of each theatre

b. OLTP

```

1 SELECT t.name AS Theatre,
2      EXTRACT(YEAR FROM pf.pdate) AS Year, TO_CHAR(pf.pdate, 'Month') AS Month, SUM(tp.totalamount) AS Monthly_Sales
3 FROM ops$yyang00.theatre t
4 JOIN ops$yyang00.performance pf ON t."THEATRE#" = pf."THEATRE#"
5 JOIN ops$yyang00.ticketpurchase tp ON pf."PER#" = tp."PER#"
6 GROUP BY t.name, EXTRACT(YEAR FROM pf.pdate), TO_CHAR(pf.pdate, 'Month')
7 ORDER BY Year, Month;

```

Results	Explain	Describe	Saved SQL	History
Vue	2019	October	296	
beeston	2019	October	298.5	
victory	2019	October	299.5	
Birstall	2019	September	374.5	
Clinton	2019	September	372.5	
Cropston	2019	September	376	
Vue	2019	September	373.5	
beeston	2019	September	375.5	
victory	2019	September	374	

102 rows returned in 0.02 seconds [Download](#)

Figure 8 OLTP Analysis of Monthly Sales of each theatre

iii. Top-spending clients by theatre

a. Data Mart

```

1 SELECT t.name AS Theatre, (c.title || '.' || c.name) AS Client, SUM(f.TotalAmount) AS Total_Spent
2 FROM group13_fact_ticketpurchase f
3 JOIN group13_dim_theatre t ON f.theatre_id = t.theatre_id JOIN group13_dim_client c ON f.client_id = c.client_id
4 GROUP BY t.theatre_id, t.name, c.title, c.name
5 HAVING SUM(f.TotalAmount) = ( SELECT MAX(SUM(f2.TotalAmount))
6 FROM group13_fact_ticketpurchase f2
7 WHERE f2.theatre_id = t.theatre_id
8 GROUP BY f2.client_id ) ORDER BY t.name;

```

Results	Explain	Describe	Saved SQL	History
Birstall		MR.Simon M4907	37.5	
Clinton		MS.Hana W4238	37.5	
Cropston		MR.Yang M131	30.5	
Victory		MR.Paco M1475	31	
Vue		MR.Liu M3663	31.5	

7 rows returned in 0.03 seconds [Download](#)

Figure 9 Data Mart Analysis to find top-spending client in each theatre

b. OLTP

```

1 SELECT t.name AS Theatre, c.name AS Client, SUM(tp.totalamount) AS Total_Spent
2 FROM ops$yyang00.theatre t
3 JOIN ops$yyang00.performance pf ON t."THEATRE#" = pf."THEATRE#"
4 JOIN ops$yyang00.ticketpurchase tp ON pf."PER#" = tp."PER#"
5 JOIN ops$yyang00.client c ON tp."CLIENT#" = c."CLIENT#"
6 GROUP BY t."THEATRE#", t.name, c.name
7 HAVING SUM(tp.totalamount) = (
8 SELECT MAX(client_total)
9 FROM (
10 SELECT tp2."CLIENT#", SUM(tp2.totalamount) AS client_total
11 FROM ops$yyang00.ticketpurchase tp2
12 JOIN ops$yyang00.performance pf2 ON tp2."PER#" = pf2."PER#" WHERE pf2."THEATRE#" = t."THEATRE#"
13 GROUP BY tp2."CLIENT#"
14 )
15 ) ORDER BY t.name;

```

Results	Explain	Describe	Saved SQL	History
beeston		Alex m33	31	
beeston		Lee m1407	31	
victory		Paco m1475	31	

7 rows returned in 0.03 seconds [Download](#)

Figure 10 OLTP Analysis to find out top spending client in each theatre

c. Data Mart after creating an index

```

1 CREATE INDEX idx_fact_analysis_theatre_client ON group13_fact_ticketpurchase (theatre_id, client_id, TotalAmount);

```

Results	Explain	Describe	Saved SQL	History
Index created.				
0.03 seconds				

Figure 11 Index creation on fact table

1	
2	SELECT t.name AS Theatre, (c.title '.' c.name) AS Client, SUM(f.TotalAmount) AS Total_Spent
3	FROM group13_fact_ticketpurchase f
4	JOIN group13_dim_theatre t ON f.theatre_id = t.theatre_id
5	JOIN group13_dim_client c ON f.client_id = c.client_id
6	GROUP BY t.theatre_id, t.name, c.title, c.name
7	HAVING SUM(f.TotalAmount) = (
8	SELECT MAX(SUM(f2.TotalAmount))
9	FROM group13_fact_ticketpurchase f2
10	WHERE f2.theatre_id = t.theatre_id
11	GROUP BY f2.client_id
12	)
13	ORDER BY t.name;
14	

Results	Explain	Describe	Saved SQL	History
---------	---------	----------	-----------	---------

THEATRE	CLIENT	TOTAL_SPENT
Beeston	MR.Alex M33	31
Beeston	MR.Lee M1407	31
Birstall	MR.Simon M4907	37.5
Clinton	MS.Hana W4238	37.5
Cropston	MR.Yang M131	30.5
Victory	MR.Paco M1475	31
Vue	MR.Liu M3663	31.5

7 rows returned in 0.02 seconds [Download](#)

Figure 12 Data Mart Analysis after index creation

5.3 Comparison of DataMart and OLTP

This section contrasts OLTP and data-mart queries their design, performance, indexing and explains how these choices affect scalability and future analysis.

- Both OLTP and Data Mart queries use JOINS rather than filtering with **WHERE** clauses. This is more efficient for large databases, as JOINS take advantage of indexed foreign keys and optimised execution plans, reducing full table scans (Connolly & Begg, 2015). On the other hand, Data Mart involves fewer joins, usually between the fact and dimension columns, while OLTP queries need to combine different operational databases like Production, Performance, TicketPurchase, and Client. This simpler design improves query speed and scales better as data grows (Kimball et al., 2008; Adamson, 2010).
- A comparison of query performance for identifying the top-spending client per theatre shows that execution time improved from 0.03 to 0.02 seconds after adding indexes (Figures 9 and 12). Although modest for current data volumes, this indicates the potential for greater gains as the dataset increases (Inmon, 2005).
- Indexing in OLTP systems can slow frequent inserts and updates. In a Data Mart, indexes can be disabled during ETL and rebuilt afterwards so bulk loads remain efficient.

Overall, the dimensional design and indexing of the Data Mart support faster queries, simpler large-scale analysis, and better scalability, making it well suited to long-term reporting and decision support.

References

Adamson, C. (2006). Mastering Data Warehouse Aggregates: Solutions for Star Schema Performance. John Wiley & Sons.

Adamson, C. (2010). Star Schema: The Complete Reference. McGraw-Hill Osborne.

Connolly, T. & Begg, C. (2015). Database Systems: A Practical Approach to Design, Implementation and Management (6th ed.). Pearson.

Inmon, W.H. (2005). Building the Data Warehouse. Wiley Publishing Inc.

Kimball, R., Ross, M., Thornthwaite, W., Mundy, J. & Becker, B. (2008). The Data Warehouse Lifecycle Toolkit (2nd ed.). Wiley Publishing Inc.

Lucid Software Inc. (2025). Lucidchart [Diagramming application]. Retrieved from <https://www.lucidchart.com>

Bibliography

Stackowiak, R. (2007). Oracle Data Warehousing. Wiley Publishing Inc.

Griesemer, B. (2011). Oracle Warehouse Builder 11g R2: Getting Started. Packt Enterprise.